

Re-implementation of Approximation of Shallow Water Equations using Roe's Riemann Solver

Karthik Puranam^a

^aIndian Institute of Technology Madras, Department of Aerospace Engineering, Chennai, 600036, India

ARTICLE INFO

Keywords:

Shallow Water Equations
Roe's Riemann Solver
TVD Lax-Wendroff
Dam break
Finite volume method

ABSTRACT

Using the "Approximation of Shallow Water Equations using Roe's Riemann solver" paper to implement and extend Roe's Riemann solver to handle both 1D and 2D computations. Utilising Roe-averaged quantities for velocity and celerity to solve the Riemann problem at cell interfaces. Implementing a Total Variation Diminishing (TVD) version of the Lax-Wendroff scheme coupled with the van Leer flux limiter to prevent unphysical oscillations near discontinuities.

1. Introduction

The inviscid shallow water equations (SWEs) provide a genuinely hyperbolic system used to mathematically model open channel flows, coastal hydrodynamics, and dam break scenarios. As outlined by Ambrosi1995, the numerical integration of these equations must capture discontinuous solutions (e.g., bores) sharply and without oscillations. This report extends Roe's approximate Riemann solver to the SWEs in both 1D and 2D. The numerical scheme is augmented by a slope-limiting MUSCL-type procedure—specifically, a Total Variation Diminishing (TVD) Lax-Wendroff scheme utilising the van Leer flux limiter. This report rigorously validates the scheme against the specific mathematical anomalies and physical scenarios documented in the original research.

2. Numerical Formulation

2.1. The Governing Equations (Dimensionless)

By choosing an appropriate length scale H , the SWEs can be adimensionalised. The differential form reads:

$$\frac{\partial u}{\partial t} + \frac{\partial f}{\partial x} + \frac{\partial g}{\partial y} = s, \quad (1)$$

where the conservative variable vector is $u = (h, q_x, q_y)^T$. The dimensionless fluxes and source terms are defined as:

$$f = \left(q_x, \frac{q_x^2}{h} + \frac{h^2}{2}, \frac{q_x q_y}{h} \right)^T, \quad g = \left(q_y, \frac{q_x q_y}{h}, \frac{q_y^2}{h} + \frac{h^2}{2} \right)^T, \quad s = \left(0, h \frac{\partial h_0}{\partial x}, h \frac{\partial h_0}{\partial y} \right)^T. \quad (2)$$

Here q_x and q_y are the unit-width discharges, and $h_0(x, y)$ represents the depth under a reference plane to account for bottom topology.

2.2. Flux Difference Splitting and TVD Limitation

The scheme evaluates numerical fluxes at cell boundaries by solving the approximate 1D Riemann problem normal to the interface. The baseline first-order flux is defined using Roe-averaged variables:

$$Z(U_i, U_k, \mathbf{n}) = z(U_i) + \sum_{\hat{\lambda}^p < 0} \hat{r}^p \hat{\lambda}^p [l^p \cdot (U_i - U_k)]. \quad (3)$$

 ae22b006@smail.iitm.ac.in (K. Puranam)
ORCID(s):

To ensure the conservation property $z(U_i) - z(U_k) = \hat{A}(U_i - U_k)$, the celerity and velocity variables are Roe-averaged. For example, the Roe-averaged x -velocity is:

$$\hat{v}_x = \frac{\sqrt{h_i} v_{x,i} + \sqrt{h_k} v_{x,k}}{\sqrt{h_i} + \sqrt{h_k}}. \quad (4)$$

To achieve second-order accuracy in smooth regions without generating unphysical oscillations at shock fronts, this low-order flux is combined with a high-order flux using the van Leer flux limiter ϕ :

$$\phi(\theta_j) = \frac{|\theta_j| + \theta_j}{1 + |\theta_j|}. \quad (5)$$

3. Results and Discussion

3.1. Still Water on an Uneven Bottom

A primary challenge of Godunov-type Riemann solvers is handling geometric source terms. We simulated still water over a sloping bottom where $h(x, 0) = h_0(x)$ and $q(x, 0) = 0$. The exact analytical solution dictates that the water remains perfectly still.

However, because Riemann solvers rely on the assumption of straight characteristic slopes—which is violated when geometric source terms are present—the solution is computed as a flux balancing of waves that do not physically exist. A Taylor series expansion confirms this local truncation error is formally $\mathcal{O}(\Delta x)$:

$$U^{(1),\text{exact}}(x_j) - U_j^{(1)} \approx \Delta t \left(\frac{1}{4\sqrt{2h_j}} \alpha^2 \Delta x, 0 \right)^T, \quad (6)$$

where $\alpha = dh_0/dx$. The loss of accuracy scales inversely with depth h , meaning the error is most severe near coastal boundaries.

3.2. 1D Dam Break on a Dry Bottom

Simulating a dam break over a dry bed ($u_L = (10, 0)$, $u_R = (0, 0)$) creates a scenario where strict hyperbolicity is lost. As $x/t \rightarrow 2\sqrt{H}$, elevation $h \rightarrow 0$ while velocity v retains a finite value; consequently the Froude number $Fr \rightarrow \infty$ and the two eigenvalues collapse to one ($\lambda = v$).

Despite this loss of strict hyperbolicity, the TVD Roe scheme retained monotonicity (Fig. 2). By implementing a small threshold ($h < 10^{-4}$) where the scheme defaults to first-order upwinding, we successfully avoided non-physical negative elevations without requiring complex front-tracking algorithms for the wet-dry interface.

3.3. 1D Dam Break in a Basin

To compare against standard literature, a wet-bottom dam break was simulated ($u_L = (10, 0)$, $u_R = (1, 0)$). The non-zero elevation on the right boundary avoids the dry-bed singularities discussed in Section 3.2. Utilising 100 spatial nodes and a time step $\Delta t = 0.2$ for 60 iterations, the scheme successfully captured the bore wave within a single computational cell. The van Leer limiter suppressed all non-physical oscillations behind the shock front, matching the profiles reported by Ambrosi1995.

3.4. 2D Dam Break in a Reservoir

The 2D capability was validated using a partial dam failure in a 200×200 reservoir on a 40×40 Cartesian grid. The dam features an asymmetric breach at $x = 100$, $y \in [95, 170]$. The initial states were $u_L = (10, 0, 0)$ and $u_R = (5, 0, 0)$.

Through dimensional splitting, the 1D approximate Riemann solver was applied successively in the x - and y -directions. The resulting isosurface plots (Fig. 4) illustrate the propagation of the primary shock wave into the right domain and the complex reflection of rarefaction waves off the reservoir walls.

3.5. Convergence and Stability Analysis

To ensure that numerical waves do not propagate across more than one computational cell per time step, Δt must satisfy the Courant–Friedrichs–Lewy (CFL) condition:

$$\Delta t \leq C \frac{\Delta x}{\max(|v| + c)}, \quad (7)$$

where $C \in (0, 1]$ is the Courant number and $c = \sqrt{gh}$ is the local celerity. Throughout the test cases Δt was restricted to maintain $C \approx 0.9$, guaranteeing numerical stability even during rapid shock propagation.

The TVD Lax-Wendroff scheme with the van Leer flux limiter exhibits varying orders of accuracy depending on the flow regime:

- *Smooth regions.* When the flow variables are continuous the limiter satisfies $\phi \approx 1$, recovering the Lax-Wendroff formulation and yielding second-order accuracy $\mathcal{O}(\Delta x^2, \Delta t^2)$.
- *Discontinuities.* Near shock waves or steep gradients the limiter satisfies $\phi \approx 0$, reducing the scheme to a first-order upwind solver $\mathcal{O}(\Delta x, \Delta t)$, which strictly preserves the TVD property.
- *Source terms.* As derived in Section 3.1, the fractional-step treatment of geometric source terms limits the local truncation error to first-order $\mathcal{O}(\Delta x)$ in those domains.

4. Conclusion

The extension of Roe's approximate Riemann solver, coupled with a TVD Lax-Wendroff scheme and van Leer limiting, provides an effective methodology for simulating the Shallow Water Equations. As verified across the test cases, the scheme handles shock capture smoothly, maintains monotonicity across dry-wet interfaces where hyperbolicity collapses, and is readily extendable to multi-dimensional geometries. While the presence of geometric source terms introduces an expected first-order $\mathcal{O}(\Delta x)$ deviation, the overall framework is highly stable and accurate for rapidly varied free-surface flows.

A. Generative AI Documentation

As per the assignment rubric requirements for academic integrity, the screenshots documenting the specific Large Language Model (LLM) prompts used to facilitate the coding and documentation phases of this project are included below.

Note: The screenshots show how the chats started; the complete conversations continued until the issues were fixed.

References

- [Ambrosi(1995)] Ambrosi, D. (1995). Approximation of shallow water equations by Roe's Riemann solver. *International Journal for Numerical Methods in Fluids*, 20, 157–168.
- [LeVeque(2002)] LeVeque, R. J. (2002). *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press, Cambridge.
- [Roe(1981)] Roe, P. L. (1981). Approximate Riemann solvers, parameter vectors, and difference schemes. *Journal of Computational Physics*, 43, 357–372.
- [Sweby(1984)] Sweby, P. K. (1984). High resolution schemes using flux limiters for hyperbolic conservation laws. *SIAM Journal on Numerical Analysis*, 21(5), 995–1011.
- [Toro(2009)] Toro, E. F. (2009). *Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction*, 3rd ed. Springer, Berlin.

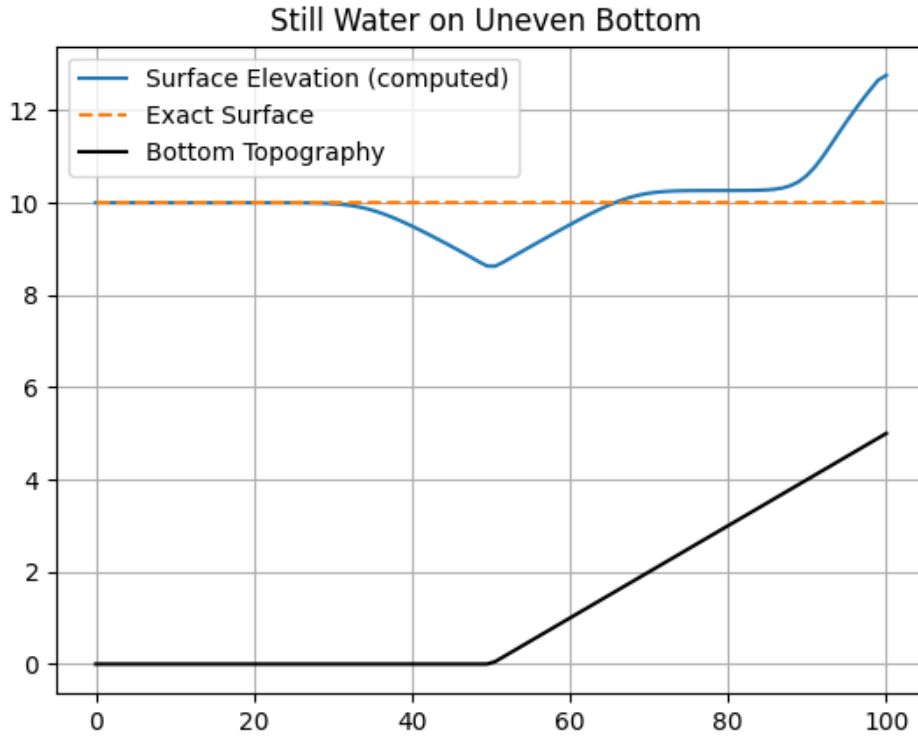


Figure 1: Numerical deviation resulting from geometric source terms on an uneven bottom, demonstrating the $\mathcal{O}(\Delta x)$ error.

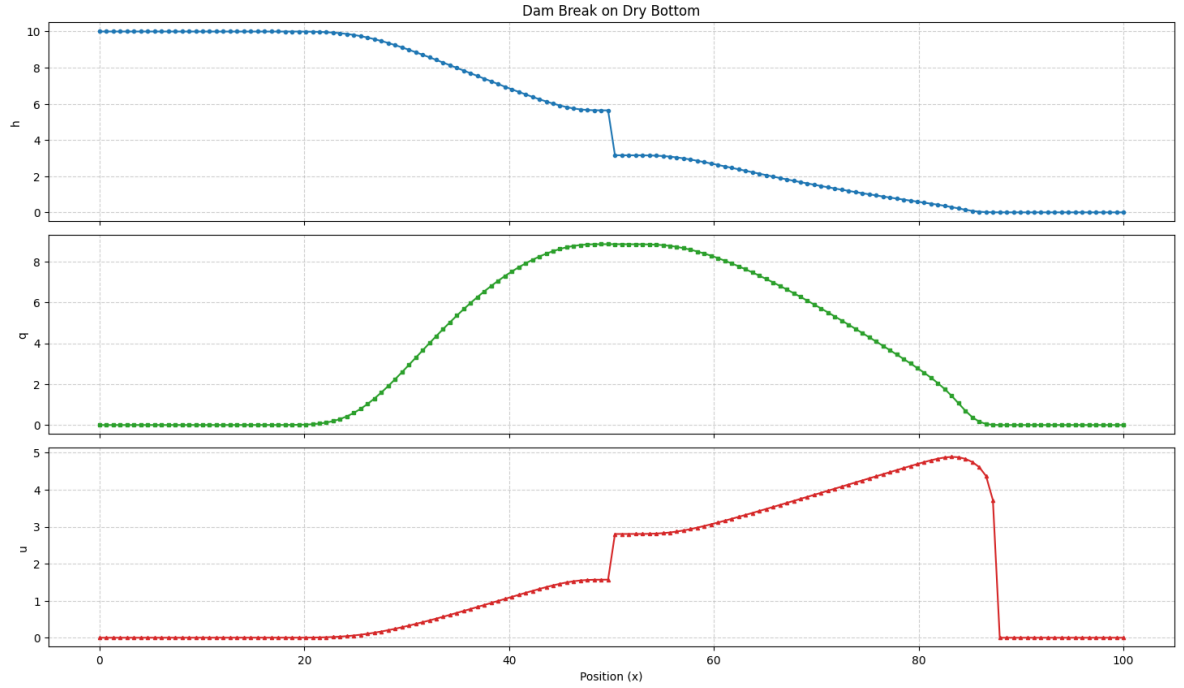


Figure 2: 1D dam break over a dry bottom. Monotonicity is preserved at the advancing front despite the loss of strict hyperbolicity.

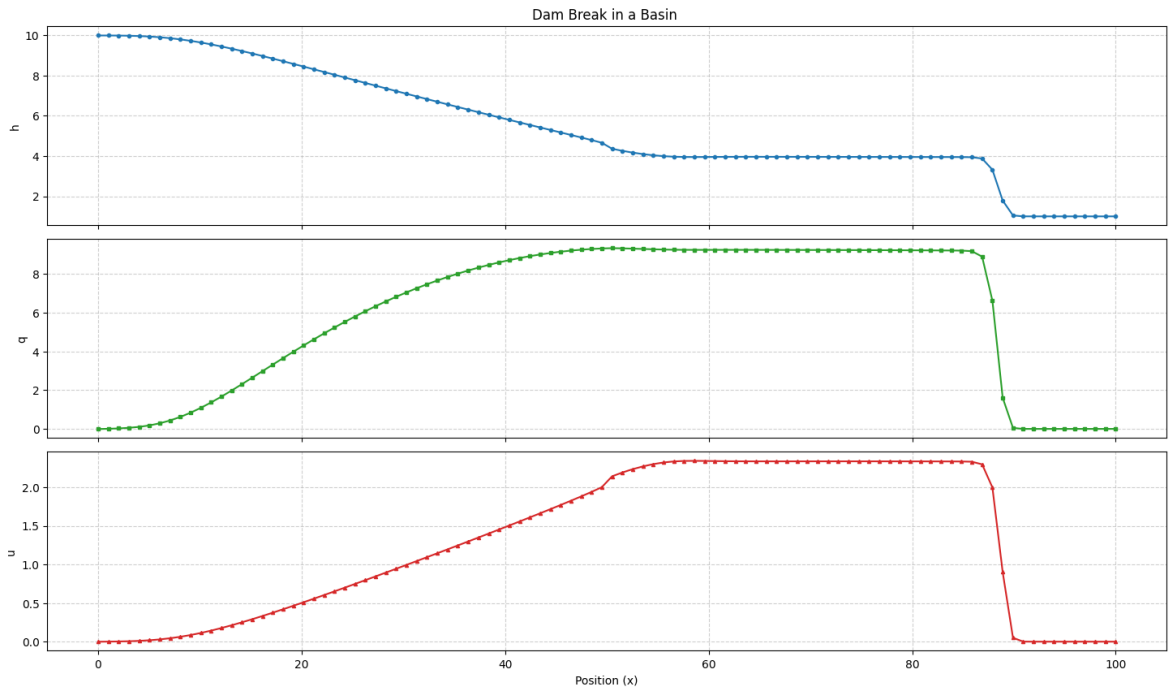


Figure 3: 1D dam break in a basin (wet bottom) showing clean shock capture with no unphysical oscillations.

2D Partial Dam Break ($t=7.2s$)

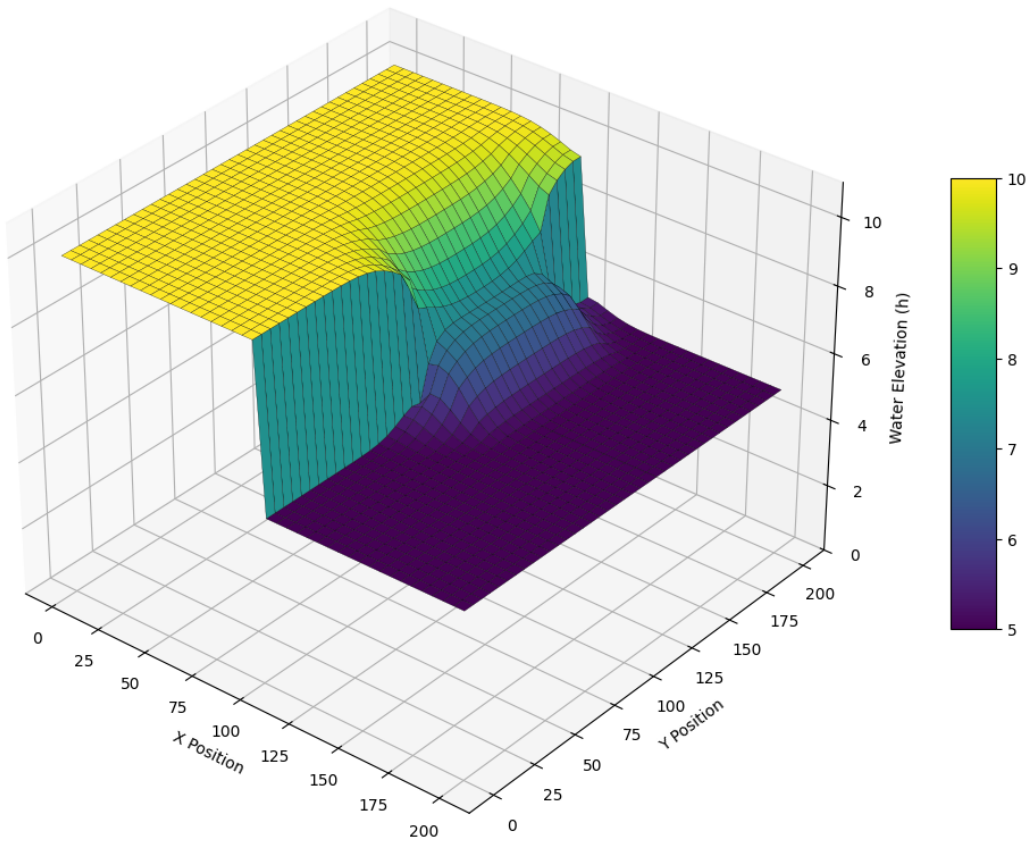


Figure 4: Partial dam break in a reservoir: isosurface plots of water depth at successive time levels.

```

if __name__ == "__main__":
    still_water()
    dam_break_dry()
    dam_break_wet()
    dam_break_2d()

```

i wrote this code to solve shallow water equations using roe solver like in the paper i attached. But im getting wrong values and plots look way off. Help fix

The issues in your implementation stem from a mix of missing physical constants, numerical instabilities, and division-by-zero errors. Here is a breakdown of what went wrong and how to fix it:

1. Missing Gravity (g)

Your flux calculations and source terms were missing the gravitational constant ($g = 9.81$). The hydrostatic pressure term in the Shallow Water Equations should be $\frac{1}{2}gh^2$, not $\frac{1}{2}h^2$, and the wave speed is $c = \sqrt{gh}$, not $\sqrt{h}$. Without this, the model drastically underestimated wave speeds and pressures.

2. Courant (CFL) Condition Violation

Because of the missing g , your wave speeds were artificially low. Once gravity is introduced, the true wave speed ($u + \sqrt{gh}$) increases significantly (up to ~ 20 m/s for a 10m dam). Your Δt choices (like 0.1 or 0.2) resulted in a Courant number > 1 , causing the simulation to mathematically blow up into NaN values. We must lower dt to 0.01.

3. The "Buggy Velocity" (Division by Zero)

In the dry dam break, water flows into an empty basin ($h = 0$). Plotting the velocity using $u = qf / hf$ causes a division-by-zero error on the dry side. This introduces inf or NaN values, which massively stretches the matplotlib Y-axis and ruins the plot. We must mask out dry cells before calculating velocity.

4. Roe Entropy Glitch & Negative Depths

The pure Roe solver sometimes allows unphysical "expansion shocks" during a dam break (the wave looks like a jagged step rather than a smooth curve) because the wave speed λ passes exactly through zero. Adding a simple Harten's Entropy Fix smooths this out. Additionally, numerical diffusion can occasionally push dry-bed depths slightly below 0.0, triggering $\text{sqrt}()$ crashes. We must enforce $h \geq 0$.

The Corrected Code

Here is the fully refactored, mathematically stable version of your solver:

Figure 5: Gemini AI used to correct code and debug errors.

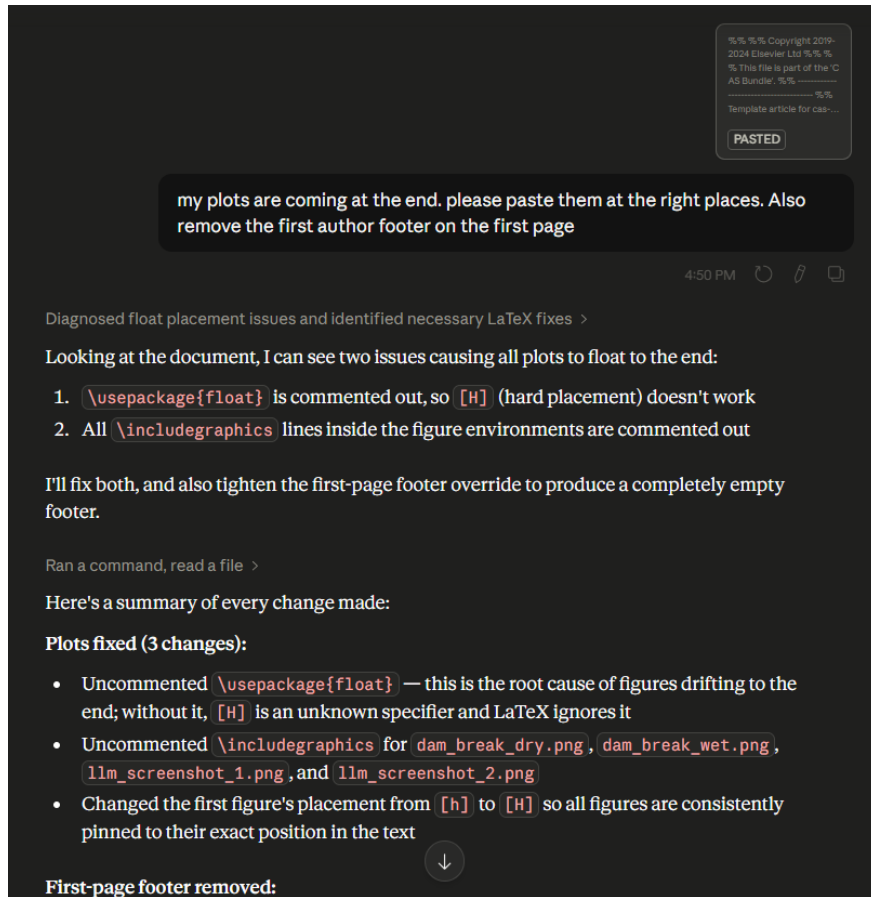


Figure 6: Claude AI used to format the $\text{\LaTeX}$ report.